



Departamento de Computación
Asignatura: Ingeniería de Software I (3385)
Año 2025

PRÁCTICO 2B:
EJERCICIO TRANSVERSAL INTEGRADOR:
SISTEMA DE GESTIÓN ESTUDIANTEL

En el Práctico 1B, cada equipo realizó un análisis del sistema de gestión estudiantil, generando los requerimientos iniciales, un diagrama de clases de contexto y un backlog. Dando continuidad a eso, el Práctico 2B se enfoca en aplicar conceptos de diseño de software y comenzar la implementación, utilizando un código base que les proporcionaremos.

Objetivos del práctico:

Que las/os estudiantes puedan:

- Comprender y analizar una arquitectura de software basada en un código base existente.
- Identificar y explicar patrones de diseño implementados en un sistema real (Ingeniería Inversa).
- Diseñar y modelar flujos de interacción a nivel de componentes tecnológicos. ● Implementar funcionalidades específicas del backlog priorizado e integrarlas al código existente.
- Consolidar la colaboración en equipo y el uso de herramientas de control de versiones.

Diseño, patrones y primeros pasos en la Implementación

En esta etapa es fundamental que comprendan la diferencia entre la metodología Scrum, que nos indica *cómo* organizar y gestionar el trabajo, y el *set de herramientas* o tecnologías, que se usan para llevarlo a cabo.

El código base que proporcionaremos desde la asignatura incluye la mayoría de las *herramientas* ya configuradas y funcionando para las funcionalidades básicas de login (inicio de sesión) y gestión de usuarios.

Actividades a realizar

- **Preparación del Código Base:** Desde el repositorio de la asignatura https://github.com/marcelouva/is1_2025_eti.git , cada equipo deberá realizar un **fork** del proyecto base del sistema de gestión estudiantil, para crear una copia independiente del código en su propio repositorio.

Este repositorio incluye la estructura inicial del proyecto en Java, las configuraciones de Maven y una versión funcional del sistema de login y carga básica de usuarios.

Luego, cada integrante del equipo deberá clonar ese repositorio en su entorno local y verificar que el proyecto compile y se ejecute correctamente.

- **Análisis del código fuente y patrones de diseño:** Dentro del código base provisto, deberán identificar y analizar el uso de un patrón de diseño que ya se encuentra implementado.

¿Qué patrón identificaron?

El patrón utilizado es un **Singleton**.

¿Dónde y cómo se aplica en el código?

Este PD es utilizado en la clase App, mediante el DBConfigSingleton. El método getInstance() devuelve una única instancia compartida, dicha instancia contiene los parámetros de conexión que luego se usan en cada petición mediante el Base.open(...). Por lo tanto, todas las partes de la aplicación utilizan la misma configuración centralizada de conexión, sin la necesidad de utilizar múltiples objetos. Cada solicitud de Spark abre una conexión a la base de datos usando la misma configuración compartida, sin la necesidad de crear nuevos objetos de configuración.

¿Qué problema resuelve este patrón en ese contexto?

El Singleton utilizado garantiza una única instancia global y consistente, simplifica el mantenimiento en casos de cambios de url, es decir se modifica en un solo lugar, y facilita el acceso centralizado a los parámetros de conexión desde cualquier parte del código. En conclusión el patrón utilizado garantiza que toda la aplicación use una única fuente coherente de parámetros de conexión a la base de datos.

- **Implementación de Historias de Usuario (HU):** Cada equipo deberá implementar la siguiente Historia de Usuario en el proyecto.

ID de HU	001
Título	Alta de profesor al sistema
Declaración	Como administrador del sistema, quiero registrar un nuevo profesor ingresando su información personal, para poder asignarlo a las asignaturas correspondientes dentro de una carrera.
Descripción Detallada	Una vez iniciada la sesión se mostrará un panel de control con diferentes opciones de selección, una de ellas es “alta de profesor”. Al seleccionar el profesor se deberá registrar un nuevo profesor ingresando nombre, apellido, dni, carrera y email. Si al ingresar datos que no pueden ser duplicados como el dni o correo (que son únicos) se desplegará un mensaje de error que indica que estos campos ya están asociados a otra persona y después se reiniciarán los campos para volver a completarlos. En caso de que el email introducido no corresponda con el formato de correo electrónico se extenderá un cuadro de texto que indica el problema.
Criterios de Validación (Criterios de Aceptación)	● Flujo exitoso: Al completar todos los campos obligatorios (nombre, apellido, correo, DNI) con datos válidos y guardar, el sistema muestra un mensaje de éxito. ● Validaciones de Datos: El sistema debe impedir el registro si: ○ Faltan campos obligatorios. ○ Si el formato del correo electrónico no es válido. ○ El correo electrónico o el DNI ya existen en la base de datos. ● Manejo de Errores: Si alguna validación falla, el sistema debe

	mostrar un mensaje de error claro, sin permitir que se guarde el formulario. ● Acción de Cancelar: El formulario debe incluir un botón "Cancelar" que elimine todos los datos ingresados y devuelva al usuario a la pantalla anterior.
Tareas Asociadas a la Implementación	- Desarrollar interfaz del profesor. - Crear tabla para la base de datos. - Generar mensajes de confirmación: "Profesor " + nombre + " " + apellido + " registrado exitosamente!". - Generar mensaje de impedimentos: "El DNI ya está registrado en el sistema". "El email ya introducido ya existe". "Incluye un signo "@" en la dirección de correo electrónico. La dirección " " no incluye el signo "@".

Formato de Entrega (en GitHub):

Todo el progreso y la documentación de este práctico deben mantenerse en el repositorio de GitHub del equipo. Actualicen la carpeta **documentación** con un archivo llamado **PRACTICO_2_DOCUMENTACION**, que incluya la explicación del patrón de diseño preimplementado.